

LAPORAN TUGAS AKHIR UAS

Perancangan dan Implementasi Sistem Robotika Berbasis Intelligent Systems



Disusun Oleh (Kelompok UAS):

Nama	NIM
Muhammad Khalis Farhan Azvi	— 22/493199/TK/54016
Valen Fatli	— 22/493755/TK/54109
Yasir Abdulaziz	— 22/497079/TK/54468
Galih Salman Sadewo	— 22/498121/TK/54625
Faris Afif Ridyasmara	— 22/502798/TK/54902

Dosen Pengampu: Dr.Eng. Dwi Joko Suroso, S.T., M.Eng.

Instrumentasi Sistem Robotika

Departemen Teknik Nuklir dan Teknik Fisika

Fakultas Teknik

Universitas Gadjah Mada

Tahun 2026

Daftar Isi

1	Introduction	1
2	Tinjauan Pustaka	4
3	Perancangan Sistem	5
3.1	Perancangan Perangkat Keras (Hardware)	5
3.2	Perancangan Sistem Visual (Visual)	6
3.2.1	Akuisisi Citra dan Manajemen Kamera (<i>camera_manager.py</i>) . .	6
3.2.2	Prapemrosesan dan Segmentasi Warna (<i>image_processor.py</i>) . . .	8
3.2.3	Deteksi Kontur Objek (<i>object_detector.py</i>)	10
3.2.4	Estimasi Jarak Visual (<i>distance_estimator.py</i>)	11
3.2.5	Integrasi dan Kalibrasi Layanan (<i>calibration_service.py</i> & <i>stream_service.py</i>)	13
3.3	Perancangan Logika Kontrol dan Navigasi (Logic)	15
3.3.1	Pengendali Driver Motor (<i>motor_manager.py</i>)	16
3.3.2	Pengelola Sensor Jarak Ultrasonik (<i>ultrasonic_manager.py</i>)	18
3.3.3	Algoritma Kontrol PID dan Logika Navigasi Utama (<i>navigator.py</i>)	20
4	Pengujian dan Hasil Pembahasan	29
4.1	Pengujian Akurasi Sensor Jarak	29
4.2	Analisis Pengujian Navigasi	29
5	Kesimpulan dan Saran	30
5.1	Kesimpulan	30
5.2	Saran	30

Daftar Gambar

1	Diagram Blok Arsitektur Perangkat Keras Robot.	5
2	Dokumentasi Fisik Robot Hasil Perakitan Kelompok.	6

Daftar Tabel

1	Data Pengujian Akurasi Sensor Jarak Ultrasonik HC-SR04.	29
---	---	----

1 Introduction

Autonomous mobile robots are increasingly used in indoor applications such as material transportation, inspection, surveillance, object retrieval, healthcare, and educational robotics. To operate autonomously, a mobile robot must be able to perceive its surroundings, identify a desired destination, determine an appropriate direction of motion, and avoid collisions with nearby objects. These functions depend strongly on the sensing system because environmental information is required for object recognition, distance measurement, motion planning, and robot control [1].

Different sensing technologies can be used to support autonomous navigation, including cameras, ultrasonic sensors, infrared sensors, LiDAR, and depth cameras. A monocular camera is particularly attractive for low-cost robots because it can provide information regarding object color, apparent size, shape, and position within the robot's field of view. Recent studies have demonstrated that monocular vision can be used to recognize obstacles and produce navigation commands for mobile robots operating in indoor environments [2]. Vision-based systems can also perform real-time obstacle detection and path generation without relying on expensive LiDAR or depth-sensing devices [3].

Despite these advantages, a monocular camera does not directly measure physical distance. The apparent dimensions of an object in an image depend on its actual dimensions, camera parameters, viewing angle, and distance from the camera. Therefore, the position and dimensions of a bounding box can indicate the relative direction and proximity of an object, but additional calibration is required to convert these visual features into an estimated physical distance.

Color-based object detection also presents practical challenges. Changes in illumination, shadows, reflections, and camera exposure can shift the pixel values representing the same physical color. Consequently, permanently defined color thresholds may not provide consistent detection under different environmental conditions. A recent target-recognition study using monocular vision showed that HSV-based segmentation and threshold calibration under the actual experimental illumination can improve the separation of a colored target from the background [4]. This approach is particularly relevant when the target color is only specified immediately before the robot begins its task.

Ultrasonic sensing provides a low-cost method for obtaining direct distance measurements. A single ultrasonic sensor can be used to detect frontal obstacles, determine whether an object has entered a critical range, and support collision-avoidance decisions [5]. However, ultrasonic sensing alone cannot reliably determine the color or visual identity of an object. Its measurement is also limited by the sensor's field of view, object

orientation, and surface characteristics. Therefore, camera and ultrasonic measurements provide complementary information: the camera identifies and localizes objects in the image, while the ultrasonic sensor supplies frontal distance information.

Existing vision-based navigation studies commonly employ convolutional neural networks, semantic segmentation, geometric models, or mapping algorithms [2, 3]. Although these methods can provide accurate environmental perception, they may require considerable computational resources, training data, or additional system complexity. Such requirements can limit their implementation on embedded platforms such as the Raspberry Pi 3 Model B. Furthermore, many systems assume that the target characteristics are known before deployment, whereas the target color in the considered competition is determined at the test location.

This study develops a two-wheeled mobile robot capable of autonomously navigating toward a color-defined target while avoiding obstacles on a structured indoor track. The robot uses a webcam as the primary visual sensor, an ultrasonic distance sensor as the frontal distance sensor, and a Raspberry Pi 3 Model B as the processing and control unit. Image acquisition, color segmentation, contour extraction, and bounding-box generation are implemented using the OpenCV library.

Before autonomous navigation begins, the target color is calibrated under the actual lighting conditions of the competition environment. The calibrated color thresholds are then used to segment the target and relevant obstacle regions from the camera frames. The detected regions are represented by bounding boxes. The horizontal center of each bounding box is used to determine whether an object is located on the left, center, or right side of the robot's field of view.

The dimensions of the bounding box are used as visual features for estimating object proximity. During the distance-calibration procedure, the bounding-box measurements are paired with reference distance measurements obtained from the HC-SR04 sensor. This procedure establishes a relationship between the apparent dimensions of the detected object and its physical distance from the robot. During navigation, the ultrasonic sensor also provides an additional safety measurement when an obstacle is located directly in front of the robot.

Based on the detected target position, the estimated distance and the obstacle information, the navigation algorithm determines the required motion command. The available commands include moving forward, turning left, turning right, stopping, and performing an obstacle-avoidance maneuver. After avoiding an obstacle, the robot redirects its movement toward the detected target until it reaches the designated target area.

The objective of this study is to design, implement and evaluate an integrated perception and navigation system that enables a two-wheeled mobile robot to reach a color-defined target without colliding with obstacles. The main contributions of this work are summarized as follows:

1. On-site color-calibration procedure that enables the robot to adapt to a target color specified immediately before operation;
2. Lightweight OpenCV-based object-detection method that represents detected targets and obstacles using bounding boxes;
3. Ultrasonic-assisted calibration procedure that relates bounding-box dimensions to physical object distance;
4. Integrated navigation strategy that combines target-directed movement and reactive obstacle avoidance; and
5. Implementation using a webcam, an ultrasonic distance sensor, and a Raspberry Pi 3 Model B as a low-cost embedded robotic platform.

The proposed system emphasizes computationally lightweight image processing and low-cost sensing instead of computationally intensive perception models or expensive depth sensors. Its performance is evaluated on a structured indoor track considering target and obstacle detection accuracy, distance-estimation error, maneuverability, and the success rate of reaching target color during the experiment.

2 Tinjauan Pustaka

Autonomous navigation requires robust obstacle avoidance mechanisms, which have been widely addressed in recent literature through various advanced control strategies. For instance, Yan et al. propose a multi-layered approach for unmanned vehicles that combines the A* algorithm for global static path planning with the Velocity-Obstacle (VO) method to dynamically evade moving targets [6]. This highlights the necessity of hybrid, global-local systems to ensure navigational safety in complex dynamic environments.

Other studies focus on optimizing real-time reactive avoidance models. Zhang et al. introduce an improved Artificial Potential Field (APF) method that incorporates the velocity information of moving obstacles to prevent unreasonably long evasion paths, while also solving the unreachable target problem common in traditional APF algorithms [7]. Furthermore, for high-speed driving scenarios, Kim et al. utilize a Real-Time Output Constraints Model Predictive Control (RCMPC) algorithm, which optimizes the avoidance path to significantly reduce computation time while ensuring safety and smooth maneuverability [8].

Building upon these foundational concepts of real-time perception and reactive control, the robotic system described in the main project report implements an efficient, low-cost alternative for indoor autonomous navigation. Rather than relying on computationally heavy path-planning algorithms, it successfully integrates a monocular webcam for color-based target tracking and an ultrasonic sensor for direct frontal obstacle detection. By feeding this sensory data into a proportional-integral-derivative (PID) controller and a sequential finite state machine, the robot can dynamically pursue its target and reactively steer clear of obstacles with minimal computational overhead.

3 Perancangan Sistem

Pada bab ini dijelaskan arsitektur perancangan sistem robotika secara menyeluruh, yang terbagi menjadi perancangan perangkat keras (hardware), perancangan sistem visual (visi komputer), dan perancangan logika kontrol navigasi (logic).

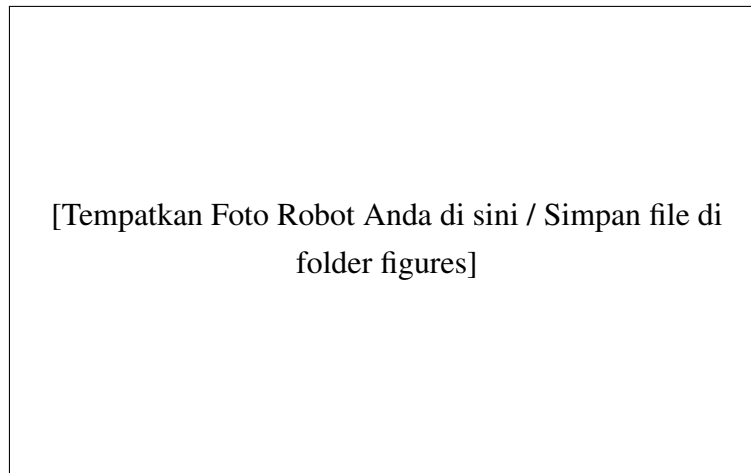
3.1 Perancangan Perangkat Keras (Hardware)

Robot ini dirancang menggunakan platform *Single Board Computer* (SBC) berupa Raspberry Pi sebagai unit pemroses utama. Perangkat keras robot terbagi menjadi komponen mekanik sasis, sistem penggerak aktuator berupa motor DC dengan driver motor L298N, sensor jarak ultrasonik HC-SR04 untuk deteksi rintangan fisik jarak dekat, serta USB Webcam sebagai sensor penglihatan utama. Diagram blok sistem perangkat keras dapat dilihat pada Gambar 1.



Gambar 1: Diagram Blok Arsitektur Perangkat Keras Robot.

Dokumentasi foto robot yang telah dirakit diletakkan pada Gambar 2. File gambar disimpan di dalam folder `figures/`.



Gambar 2: Dokumentasi Fisik Robot Hasil Perakitan Kelompok.

3.2 Perancangan Sistem Visual (Visual)

Perancangan subsistem visual berfokus pada pemrosesan citra digital yang ditangkap oleh USB Webcam secara real-time. Kamera diposisikan di bagian depan robot untuk menangkap kondisi lingkungan secara langsung. Alur pemrosesan sistem visual ini terbagi menjadi empat tahap utama: akuisisi citra, prapemrosesan dan segmentasi warna (masking), deteksi kontur objek, serta estimasi jarak visual.

3.2.1 Akuisisi Citra dan Manajemen Kamera (*camera_manager.py*)

Citra mentah diperoleh menggunakan USB Webcam dengan resolusi native 320×240 piksel. Format penangkapan video disetel menggunakan kompresi MJPG (*Motion JPEG*) melalui driver V4L2 (*Video for Linux 2*) guna meminimalkan penggunaan bandwidth USB dan beban pemrosesan pada Raspberry Pi.

Untuk menghindari hambatan (*bottleneck*) pada jalannya program utama, proses pembacaan frame kamera diisolasi ke dalam utas latar belakang (*background thread*) tersendiri yang berjalan secara asinkron dengan jeda waktu tunggu (*sleep*) sebesar 0.03 detik antar pengambilan frame. Hal ini menjamin tersedianya frame terbaru yang siap diakses setiap saat oleh modul pemrosesan citra tanpa memblokir thread eksekusi utama. Kode implementasi manajemen kamera dapat dilihat pada Kode Program 1.

```
1 import cv2
2 import threading
3 import time
4 import numpy as np
5
```

```

6 class CameraManager:
7     def __init__(self, src=0):
8         self.src = src
9         # Gunakan backend V4L2 yang jauh lebih stabil di Raspberry Pi/
Linux
10        self.stream = cv2.VideoCapture(self.src, cv2.CAP_V4L2)
11
12        # Set format ke MJPG untuk menghemat bandwidth USB di Raspberry
Pi
13        self.stream.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'
MJPG'))
14        self.stream.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
15        self.stream.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
16
17        self.grabbed = False
18        # Inisialisasi dengan frame hitam agar tidak None
19        self.frame = np.zeros((240, 320, 3), dtype=np.uint8)
20        self.stopped = False
21
22        if self.stream.isOpened():
23            print(f"[CAM] Kamera {self.src} terdeteksi.")
24            # Ambil satu frame awal secara sinkron
25            self.grabbed, self.frame = self.stream.read()
26        else:
27            print(f"[CAM] [ERROR] Kamera {self.src} tidak ditemukan!")
28
29        def start(self):
30            # Jalankan pembacaan frame di background thread
31            threading.Thread(target=self.update, args=(), daemon=True).
start()
32            return self
33
34        def update(self):
35            while not self.stopped:
36                if not self.stream.isOpened():
37                    break
38                self.grabbed, self.frame = self.stream.read()
39                # Jeda kecil agar tidak membebani CPU
40                time.sleep(0.03)
41
42        def read(self):
43            return self.frame
44
45        def stop(self):
46            self.stopped = True

```

```

47     self.stream.release()
48
49     def switch_source(self, new_src):
50         """Mengganti input kamera secara dinamis"""
51         self.stopped = True
52         time.sleep(0.1) # Beri jeda kecil agar thread update selesai
53         self.stream.release()
54
55         self.src = new_src
56         self.stream = cv2.VideoCapture(self.src, cv2.CAP_V4L2)
57         self.stream.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'
58         MJPG'))
59         self.stream.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
60         self.stream.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
61
62         self.grabbed = False
63         self.frame = np.zeros((240, 320, 3), dtype=np.uint8)
64         self.stopped = False
65
66         if self.stream.isOpened():
67             print(f"[CAM] Berhasil beralih ke Kamera {self.src}")
68             self.grabbed, self.frame = self.stream.read()
69             self.start()
70             return True
71         else:
72             print(f"[CAM] [ERROR] Gagal membuka Kamera {self.src}")
73             # Nyalakan thread update kembali agar program tidak macet
74             self.start()
75             return False

```

Listing 1: Kode Program Modul Manajemen Kamera (camera_manager.py)

3.2.2 Prapemrosesan dan Segmentasi Warna (*image_processor.py*)

Sebelum dilakukan segmentasi warna, frame citra yang bertipe BGR (*Blue-Green-Red*) terlebih dahulu disaring menggunakan filter Gaussian Blur dengan ukuran kernel 11×11 piksel. Proses ini bertujuan untuk mereduksi derau (*noise*) frekuensi tinggi dan menghaluskan tepi objek. Selanjutnya, ruang warna citra dikonversi dari BGR menjadi HSV (*Hue, Saturation, Value*) karena ruang warna HSV lebih stabil terhadap fluktuasi intensitas cahaya lingkungan.

Segmentasi objek pada modul pemrosesan citra dilakukan dengan membagi hasil thresholding warna ke dalam dua masker biner utama:

1. **Masker Target (*mask_target*):** Masker ini memisahkan warna target berdasarkan nilai ambang batas (*threshold*) warna HSV yang telah dikalibrasi sebelumnya. Rentang nilai *threshold* didefinisikan sebagai batas bawah (HSV_{lower}) dan batas atas (HSV_{upper}). Untuk menghilangkan noise piksel biner kecil yang terisolasi, diterapkan operasi morfologi berupa erosi (*erode*) sebanyak 2 iterasi, diikuti dengan dilasi (*dilate*) sebanyak 2 iterasi.
2. **Masker Warna Tajam (*mask_colorful*):** Masker ini digunakan untuk mendeteksi semua objek di lingkungan yang memiliki saturasi warna yang tajam (seperti kertas origami atau penanda fisik lainnya). Parameter ambang batas bawah ditentukan pada nilai $HSV = [0, 100, 80]$ dan batas atas pada $HSV = [180, 255, 255]$.
3. **Masker Rintangan (*mask_obstacle*):** Benda dianggap sebagai rintangan (*obstacle*) jika memiliki karakteristik warna yang tajam tetapi warnanya bukan merupakan warna target. Masker rintangan diperoleh melalui operasi logika *bitwise* AND antara masker warna tajam dengan negasi dari masker target:

$$\text{mask_obstacle} = \text{mask_colorful} \wedge \neg(\text{mask_target}) \quad (1)$$

Kode implementasi segmentasi warna dapat dilihat pada Kode Program 2.

```

1 import cv2
2 import numpy as np
3
4 class ImageProcessor:
5     def process_masks(self, frame, lower_color, upper_color):
6         blurred = cv2.GaussianBlur(frame, (11, 11), 0)
7         hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)
8
9         # 1. Masker untuk Target (Sesuai dengan warna klik pengguna)
10        mask_target = cv2.inRange(hsv, lower_color, upper_color)
11        mask_target = cv2.erode(mask_target, None, iterations=2)
12        mask_target = cv2.dilate(mask_target, None, iterations=2)
13
14        # 2. Masker untuk Benda Berwarna Tajam (Kertas origami/binder)
15        # Kertas warna-warni biasanya memiliki nilai Saturation dan
16        Value tinggi (> 100)
17        lower_colorful = np.array([0, 100, 80])
18        upper_colorful = np.array([180, 255, 255])
19        mask_colorful = cv2.inRange(hsv, lower_colorful, upper_colorful)
20
21        mask_colorful = cv2.erode(mask_colorful, None, iterations=2)

```

```

21     mask_colorful = cv2.dilate(mask_colorful, None, iterations=2)
22
23     # 3. Masker Obstacle
24     # Benda dianggap Pengecoh JIKA warnanya tajam, TAPI BUKAN
    merupakan warna target
25     mask_obstacle = cv2.bitwise_and(mask_colorful, cv2.bitwise_not(
    mask_target))
26
27     return mask_target, mask_obstacle

```

Listing 2: Kode Program Modul Pemroses Citra (image_processor.py)

3.2.3 Deteksi Kontur Objek (*object_detector.py*)

Setelah masker biner (*mask_target* dan *mask_obstacle*) terbentuk, koordinat fisik dan dimensi objek diidentifikasi menggunakan algoritma pencarian kontur eksternal (`cv2.findContours` dengan metode aproksimasi `CHAIN_APPROX_SIMPLE`). Kontur-kontur yang ditemukan disaring berdasarkan luas area pikselnya. Kontur dengan luas area yang kurang dari ambang batas minimum (`min_area = 500` piksel) akan diabaikan guna menyaring partikel debu atau noise minor di latar belakang.

Setiap kontur valid yang lolos penyaringan dilingkupi dengan kotak pembatas tegak (*Bounding Box*) berupa parameter koordinat piksel kiri atas (x, y) serta dimensi lebar (w) dan tinggi (h) menggunakan fungsi `cv2.boundingRect`. Daftar objek terdeteksi kemudian diurutkan berdasarkan ukuran luas area bounding box ($w \times h$) secara menurun (*descending*) agar objek terbesar selalu menempati indeks pertama. Kode implementasi deteksi objek dapat dilihat pada Kode Program 3.

```

1  import cv2
2
3  class ObjectDetector:
4      def __init__(self, min_area=500):
5          # min_area adalah ukuran minimal piksel agar benda tidak
    dianggap sebagai debu/noise
6          self.min_area = min_area
7
8      def find_objects(self, mask):
9          # Cari kontur (garis luar benda putih) pada masker
10         contours, _ = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)
11
12         valid_objects = []
13         for contour in contours:

```

```

14         area = cv2.contourArea(contour)
15         # Abaikan objek yang terlalu kecil
16         if area > self.min_area:
17             # Dapatkan koordinat x, y, lebar (w), dan tinggi (h)
18             x, y, w, h = cv2.boundingRect(contour)
19             valid_objects.append((x, y, w, h))
20
21         # Urutkan objek berdasarkan luas bounding box (terbesar ke
22         terkecil)
23         valid_objects = sorted(valid_objects, key=lambda obj: obj[2] *
                                obj[3], reverse=True)
24         return valid_objects

```

Listing 3: Kode Program Modul Detektor Objek (object_detector.py)

3.2.4 Estimasi Jarak Visual (*distance_estimator.py*)

Sistem visi juga dilengkapi dengan kemampuan mengestimasi jarak fisik objek dari kamera memanfaatkan model proyeksi perspektif kamera lubang jarum (*pinhole camera model*). Hubungan antara ukuran piksel objek pada sensor kamera dengan jarak fisiknya bersifat berbanding terbalik. Jarak fisik objek (D) diestimasi menggunakan persamaan matematika berikut:

$$D = \frac{W_{\text{ref}} \times D_{\text{ref}}}{w} \quad (2)$$

di mana:

- D adalah estimasi jarak fisik objek dari kamera saat ini (dalam cm).
- W_{ref} adalah lebar objek referensi dalam satuan piksel pada gambar saat proses kalibrasi dilakukan.
- D_{ref} adalah jarak fisik nyata referensi dari kamera ke objek saat proses kalibrasi dilakukan (dalam cm).
- w adalah dimensi lebar bounding box objek dalam satuan piksel pada frame saat ini.

Nilai kalibrasi awal berupa W_{ref} dan D_{ref} disimpan secara persisten ke dalam berkas konfigurasi `kalibrasi_jarak.txt` sehingga sistem dapat langsung menggunakannya ketika dijalankan kembali. Kode implementasi estimasi jarak dapat dilihat pada Kode Program 4.

```

1 import os
2

```

```

3 class DistanceEstimator:
4     def __init__(self, file_path="kalibrasi_jarak.txt"):
5         self.file_path = file_path
6         self.reference_pixel_width = 0.0
7         self.known_distance = 0.0
8         self.is_calibrated = False
9         self.load_calibration()
10
11     def load_calibration(self):
12         if os.path.exists(self.file_path):
13             try:
14                 with open(self.file_path, "r") as f:
15                     lines = f.readlines()
16                     if len(lines) >= 2:
17                         self.reference_pixel_width = float(lines[0].
strip())
18                         self.known_distance = float(lines[1].strip())
19                         self.is_calibrated = True
20                         print(f"[JARAK] Kalibrasi dimuat: {self.
reference_pixel_width}px pada {self.known_distance}cm")
21             except Exception as e:
22                 print(f"[JARAK] Gagal memuat file kalibrasi: {e}")
23
24     def calibrate(self, known_distance, pixel_width):
25         if known_distance > 0 and pixel_width > 0:
26             self.reference_pixel_width = pixel_width
27             self.known_distance = known_distance
28             self.is_calibrated = True
29             try:
30                 with open(self.file_path, "w") as f:
31                     f.write(f"{pixel_width}\n{known_distance}\n")
32                     print(f"[JARAK] Kalibrasi disimpan: {pixel_width}px
pada {known_distance}cm")
33             except Exception as e:
34                 print(f"[JARAK] Gagal menyimpan file kalibrasi: {e}")
35             return True
36         return False
37
38     def estimate_distance(self, pixel_width):
39         if not self.is_calibrated or pixel_width <= 0:
40             return 0.0
41
42         # Rumus Jarak yang disederhanakan: D_baru = (Piksel_Awal *
Jarak_Awal) / Piksel_Baru
43         distance = (self.reference_pixel_width * self.known_distance) /

```

```

44 pixel_width
    return round(distance, 1)

```

Listing 4: Kode Program Modul Estimasi Jarak (*distance_estimator.py*)

3.2.5 Integrasi dan Kalibrasi Layanan (*calibration_service.py & stream_service.py*)

Sistem visual ini berjalan sebagai layanan web server lokal berbasis Flask API pada alamat URL `http://127.0.0.1:5000`. Layanan ini memfasilitasi interaksi pengguna untuk menentukan dan memilih objek target secara dinamis menggunakan antarmuka web.

Mekanisme pemilihan target dilakukan dengan cara berikut:

1. **Mode Discovery (Inisialisasi):** Pada mode awal ini, sistem belum mengunci warna target tertentu. Umpan kamera menampilkan tulisan panduan agar pengguna mengklik tepat pada objek target di layar.
2. **Pengambilan Sampel Warna (Klik Pengguna):** Ketika pengguna mengklik koordinat piksel (x, y) objek target di web browser, koordinat tersebut dikirimkan ke web API server melalui request POST ke endpoint `/api/calibrate-color`. Layanan `CalibrationService` kemudian mengambil area ROI (*Region of Interest*) berukuran 10×10 piksel di sekeliling titik koordinat tersebut pada frame HSV saat itu.
3. **Kalkulasi Toleransi HSV:** Nilai rata-rata Hue (H_{avg}), Saturation (S_{avg}), dan Value (V_{avg}) dari ROI dihitung dan diberi batas toleransi agar deteksi tetap stabil di bawah variasi pencahayaan:

$$H_{lower/upper} = H_{avg} \pm 15, \quad S_{lower/upper} = S_{avg} \pm 60, \quad V_{lower/upper} = V_{avg} \pm 60 \quad (3)$$

Batas toleransi ini secara otomatis disimpan sebagai parameter filter warna target aktif.

4. **Mode Tracking (Pelacakan):** Setelah warna target berhasil dikalibrasi, sistem beralih dari **Mode Discovery** ke **Mode Tracking** untuk melacak pergerakan target dan rintangan berdasarkan masker warna tersebut.

Kode implementasi dari layanan kalibrasi warna dan deteksi ini dapat dilihat pada Kode Program 5.

```

1 import numpy as np
2 import cv2

```



```

3 from robot_vision_module.distance_estimator import DistanceEstimator
4
5 class CalibrationService:
6     def __init__(self):
7         self.mode = "DISCOVERY" # Mode awal adalah mencari benda apapun
8         self.target_lower = np.array([0, 0, 0])
9         self.target_upper = np.array([180, 255, 255])
10        self.is_calibrated = False
11        self.last_discovery_objects = []
12
13        self.distance_estimator = DistanceEstimator()
14        self.last_target_bbox = None
15        self.latest_objects_data = []
16
17    def calibrate_color(self, frame, x, y):
18        h, w = frame.shape[:2]
19        if x < 0 or x >= w or y < 0 or y >= h:
20            return False
21
22        # Ambil sampel dari area kecil di sekitar titik klik (agar
23        lebih representatif)
24        y_min, y_max = max(0, y-5), min(h, y+5)
25        x_min, x_max = max(0, x-5), min(w, x+5)
26        roi = frame[y_min:y_max, x_min:x_max]
27
28        hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
29
30        # Rata-rata warna
31        if hsv_roi.size > 0:
32            avg_hsv = np.average(np.average(hsv_roi, axis=0), axis=0)
33        else:
34            return False
35
36        hue, sat, val = avg_hsv[0], avg_hsv[1], avg_hsv[2]
37
38        # Toleransi
39        lower_hue = max(0, hue - 15)
40        upper_hue = min(180, hue + 15)
41
42        lower_sat = max(50, sat - 60)
43        upper_sat = min(255, sat + 60)
44
45        lower_val = max(50, val - 60)
46        upper_val = min(255, val + 60)

```

```

47     self.target_lower = np.array([lower_hue, lower_sat, lower_val])
48     self.target_upper = np.array([upper_hue, upper_sat, upper_val])
49     self.mode = "TRACKING"
50     self.is_calibrated = True
51
52     print(f"[KALIBRASI] Klik ({x}, {y}) | AVG HSV: {avg_hsv}")
53     return True
54
55     def calibrate_distance(self, known_distance):
56         if self.last_target_bbox is not None:
57             _, _, w, _ = self.last_target_bbox
58             success = self.distance_estimator.calibrate(known_distance,
59 w)
60             if success:
61                 print(f"[KALIBRASI JARAK] Berhasil! Piksel {w}px = {
62 known_distance}cm")
63                 return success
64             return False
65
66     def reset_mode(self):
67         self.mode = "DISCOVERY"

```

Listing 5: Kode Program Layanan Kalibrasi Warna Target (calibration_service.py)

Layanan API web Flask ini mengekspos endpoint-endpoint fungsional utama berikut:

- /video_feed: Mengalirkan umpan video (*video stream*) berannotasi bounding box secara real-time ke antarmuka pengguna berbasis web menggunakan format *Multipart JPEG* (MJPEG) yang disuplai oleh `stream_service.py`.
- /api/objects: Mengembalikan data koordinat tengah sumbu-x target (x_{target}) dan rintangan visual, bounding box, serta hasil estimasi jarak visual dalam format JSON untuk diakses oleh script kemudi utama robot.

3.3 Perancangan Logika Kontrol dan Navigasi (Logic)

Bagian perancangan logika kontrol merupakan pusat pengambilan keputusan robot berbasis Raspberry Pi. Logika navigasi dirancang menggunakan bahasa pemrograman Python 3 dengan struktur modular yang terdiri dari tiga berkas program utama: `motor_manager.py`, `ultrasonic_manager.py`, dan `navigator.py`. Diagram alir atau arsitektur modular perangkat lunak navigasi ini diintegrasikan dalam satu kesatuan sistem otonom.

3.3.1 Pengendali Driver Motor (motor_manager.py)

Modul `motor_manager.py` bertanggung jawab langsung untuk mengontrol arah putaran dan kecepatan motor DC menggunakan sinyal PWM (*Pulse Width Modulation*) yang disuplai oleh Raspberry Pi melalui GPIO ke driver motor L298N. Kode implementasi modul ini dapat dilihat pada Kode Program 6.

Beberapa aspek logika penting pada modul ini meliputi:

- **Kalibrasi Bias Roda (*Left Bias*):** Karena ketidakseimbangan mekanis bawaan motor DC kiri dan kanan, diterapkan pengali bias roda kiri (`left_bias`) sebesar 1.14 pada saat inisialisasi di navigator agar robot dapat melaju lurus secara simetris.
- **Pembatasan Kecepatan Proporsional (*Speed Limiting*):** Kecepatan dikontrol dalam rentang -255 hingga 255 . Jika input kecepatan melebihi batas `max_speed_limit` (disetel pada 180), kecepatan kedua motor diturunkan secara proporsional menggunakan persamaan rasio agar arah belokan (*steering ratio*) tetap terjaga sesuai instruksi navigasi.
- **Torsi Minimum Motor:** Untuk mengatasi gesekan statis sasis robot, diterapkan logika ambang batas bawah di mana motor yang diinstruksikan bergerak akan diberikan daya minimal sebesar ± 90 .

```
1 import RPi.GPIO as GPIO
2
3 class MotorManager:
4     def __init__(self, ena=13, in1=19, in2=26, enb=12, in3=16, in4=20,
5         max_speed_limit=150, left_bias=1.0):
6         # BCM Pin Configuration
7         self.ENA = ena # PWM Left
8         self.IN1 = in1 # Dir 1 Left
9         self.IN2 = in2 # Dir 2 Left
10
11         self.ENB = enb # PWM Right
12         self.IN3 = in3 # Dir 1 Right
13         self.IN4 = in4 # Dir 2 Right
14
15         self.max_speed_limit = max_speed_limit
16         self.left_bias = left_bias
17
18         # Setup Pins
19         GPIO.setmode(GPIO.BCM)
20         for pin in [self.ENA, self.IN1, self.IN2, self.ENB, self.IN3,
21             self.IN4]:
```

```

20         GPIO.setup(pin, GPIO.OUT)
21
22         # Setup PWM
23         self.pwm_left = GPIO.PWM(self.ENA, 1000) # 1kHz frequency
24         self.pwm_right = GPIO.PWM(self.ENB, 1000)
25
26         self.pwm_left.start(0)
27         self.pwm_right.start(0)
28
29     def set_motors(self, speed_left, speed_right):
30         """
31         Set speed for both motors.
32         speed range: -255 to 255 (negative for reverse)
33         """
34         # Terapkan bias untuk motor kiri
35         speed_left = speed_left * self.left_bias
36
37         # Batasi kecepatan secara proporsional agar rasio kemudi tetap
38         # terjaga
39         # Gunakan max_speed_limit sebagai plafon absolut
40         abs_l = abs(speed_left)
41         abs_r = abs(speed_right)
42         max_val = max(abs_l, abs_r)
43
44         if max_val > self.max_speed_limit:
45             ratio = self.max_speed_limit / max_val
46             speed_left *= ratio
47             speed_right *= ratio
48
49         # --- Left Motor ---
50         if speed_left >= 0:
51             GPIO.output(self.IN1, GPIO.HIGH)
52             GPIO.output(self.IN2, GPIO.LOW)
53         else:
54             GPIO.output(self.IN1, GPIO.LOW)
55             GPIO.output(self.IN2, GPIO.HIGH)
56             speed_left = abs(speed_left)
57
58         # Convert 0-255 to 0-100% duty cycle
59         duty_left = (speed_left / 255.0) * 100
60         self.pwm_left.ChangeDutyCycle(min(duty_left, 100))
61
62         # --- Right Motor ---
63         if speed_right >= 0:
64             GPIO.output(self.IN3, GPIO.HIGH)
65             GPIO.output(self.IN4, GPIO.LOW)

```

```

64     else:
65         GPIO.output(self.IN3, GPIO.LOW)
66         GPIO.output(self.IN4, GPIO.HIGH)
67         speed_right = abs(speed_right)
68
69         duty_right = (speed_right / 255.0) * 100
70         self.pwm_right.ChangeDutyCycle(min(duty_right, 100))
71
72     def stop(self):
73         self.pwm_left.ChangeDutyCycle(0)
74         self.pwm_right.ChangeDutyCycle(0)
75         GPIO.output(self.IN1, GPIO.LOW)
76         GPIO.output(self.IN2, GPIO.LOW)
77         GPIO.output(self.IN3, GPIO.LOW)
78         GPIO.output(self.IN4, GPIO.LOW)
79
80     def cleanup(self):
81         self.stop()
82         self.pwm_left.stop()
83         self.pwm_right.stop()
84         self.pwm_left = None
85         self.pwm_right = None

```

Listing 6: Kode Program Modul Pengendali Motor (motor_manager.py)

3.3.2 Pengelola Sensor Jarak Ultrasonik (ultrasonic_manager.py)

Modul `ultrasonic_manager.py` digunakan untuk berinteraksi dengan sensor ultrasonik HC-SR04 guna membaca jarak fisik hambatan di depan robot secara langsung. Kode implementasi modul ini ditunjukkan pada Kode Program 7.

Untuk memastikan akurasi dan kecepatan respon navigasi, modul ini menerapkan beberapa metode logis berikut:

- **Moving Average dan Penyaringan Data (Filtering):** Sensor melakukan 3 kali pembacaan jarak mentah dengan jeda singkat 10 ms antar pembacaan. Nilai yang dianggap valid adalah jarak dalam rentang 2 hingga 400 cm. Pembacaan yang valid kemudian dirata-ratakan untuk menyaring penciran (*noise*).
- **Faktor Koreksi Dinamis (Kalibrasi):** Modul memuat nilai faktor koreksi dari berkas teks eksternal `faktor_koreksi.txt` saat inisialisasi. Jarak akhir yang digunakan oleh robot merupakan hasil perkalian jarak mentah rata-rata dengan faktor koreksi ini ($s_{kalibrasi} = s_{mentah} \times f_{koreksi}$).

- **Pencegahan Kegagalan (Fallback):** Jika sensor gagal mendapatkan data pembacaan yang valid, fungsi secara otomatis mengembalikan nilai 999 cm agar robot tidak berhenti mendadak akibat kesalahan sensor sementara.

```
1 import RPi.GPIO as GPIO
2 import time
3 import os
4
5 class UltrasonicManager:
6     def __init__(self, trigger_pin=17, echo_pin=27, file_kalibrasi="
    faktor_koreksi.txt"):
7         self.PIN_TRIGGER = trigger_pin
8         self.PIN_ECHO = echo_pin
9         self.FILE_KALIBRASI = file_kalibrasi
10        self.faktor_koreksi = 1.0
11
12        # Setup GPIO
13        GPIO.setmode(GPIO.BCM)
14        GPIO.setup(self.PIN_TRIGGER, GPIO.OUT)
15        GPIO.setup(self.PIN_ECHO, GPIO.IN)
16        GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
17
18        self.load_kalibrasi()
19
20    def load_kalibrasi(self):
21        if os.path.exists(self.FILE_KALIBRASI):
22            try:
23                with open(self.FILE_KALIBRASI, "r") as f:
24                    self.faktor_koreksi = float(f.read().strip())
25                    print(f"[ULTRASONIC] Faktor koreksi dimuat: {self.
    faktor_koreksi:.4f}")
26            except:
27                print("[ULTRASONIC] Gagal membaca file kalibrasi,
    menggunakan 1.0")
28
29    def ukur_jarak_mentah(self):
30        total_jarak = 0
31        pembacaan_valid = 0
32
33        for _ in range(3): # Dikurangi ke 3 pembacaan agar navigasi
    lebih responsif
34            GPIO.output(self.PIN_TRIGGER, GPIO.HIGH)
35            time.sleep(0.00001)
36            GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
37
```

```

38     waktu_mulai = time.time()
39     waktu_selesai = time.time()
40     timeout = time.time() + 0.1 # Timeout dipercepat
41
42     while GPIO.input(self.PIN_ECHO) == 0:
43         waktu_mulai = time.time()
44         if waktu_mulai > timeout: break
45
46     while GPIO.input(self.PIN_ECHO) == 1:
47         waktu_selesai = time.time()
48         if waktu_selesai > timeout: break
49
50     durasi = waktu_selesai - waktu_mulai
51     jarak = (durasi * 34300) / 2
52
53     if 2 <= jarak <= 400:
54         total_jarak += jarak
55         pembacaan_valid += 1
56         time.sleep(0.01)
57
58     if pembacaan_valid > 0:
59         return total_jarak / pembacaan_valid
60     return None
61
62     def get_jarak(self):
63         mentah = self.ukur_jarak_mentah()
64         if mentah is not None:
65             return mentah * self.faktor_koreksi
66         return 999 # Kembalikan nilai jauh jika gagal baca agar robot
        tidak stop tiba-tiba
67
68     def cleanup(self):
69         pass

```

Listing 7: Kode Program Modul Sensor Ultrasonik (ultrasonic_manager.py)

3.3.3 Algoritma Kontrol PID dan Logika Navigasi Utama (navigator.py)

Modul `navigator.py` merupakan otak navigasi otonom yang mengintegrasikan data sensor jarak ultrasonik dan data visi komputer untuk mengambil keputusan gerak secara sekuensial. Kode lengkap dari modul ini ditunjukkan pada Kode Program 8.

Algoritma Kendali PID (Proportional-Integral-Derivative) Sistem kemudi otomatis saat mendeteksi target visual menggunakan pengendali PID untuk meminimalkan deviasi arah (error) terhadap titik tengah kamera. Persamaan matematika PID yang diimplementasikan adalah:

$$e(t) = x_{\text{target}} - x_{\text{setpoint}} \quad (4)$$

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (5)$$

di mana $x_{\text{setpoint}} = 160$ piksel (nilai tengah horizontal dari resolusi lebar citra 320 piksel), dan $e(t)$ adalah error posisi horizontal target. Nilai konstanta kendali yang disetel adalah $K_p = 0.8$ untuk respon proporsional yang tanggap, $K_d = 0.05$ untuk meredam osilasi arah, dan $K_i = 0.0$ untuk menghindari akumulasi error jangka panjang. Luaran kendali $u(t)$ bertindak sebagai nilai kemudi (*steer value*) yang memodifikasi kecepatan motor kiri (v_{kiri}) dan motor kanan (v_{kanan}):

$$v_{\text{kiri}} = V_{\text{base}} + u(t) \quad (6)$$

$$v_{\text{kanan}} = V_{\text{base}} - u(t) \quad (7)$$

di mana $V_{\text{base}} = 125$ merupakan kecepatan dasar gerakan maju robot.

Arsitektur Finite State Machine (FSM) Sekuensial Logika navigasi beroperasi dalam suatu loop utama tak terbatas yang berjalan secara sekuensial dan terbagi menjadi beberapa keadaan (*states*):

1. **State 1: Deteksi Rintangan Ultrasonik Jarak Dekat:** Jarak depan diperiksa terlebih dahulu menggunakan sensor ultrasonik.
 - Jika $s \leq 15$ cm, robot akan berhenti seketika.
 - Sistem memeriksa data visi komputer. Jika objek di depannya terdeteksi sebagai TARGET dengan area visual > 10.000 piksel dan jarak fisik ≤ 12 cm, robot mendeteksi kondisi **Finish** dan menghentikan navigasi sepenuhnya.
 - Jika bukan target akhir, robot mendeteksi rintangan (OBSTACLE) fisik. Robot mundur perlahan ($-V_{\text{SLOW}}$) selama 0.5 detik, berhenti 0.2 detik, kemudian melakukan manuver berputar menghindar ke kiri atau kanan selama 0.3 detik berdasarkan arah memori target terakhir (*last_known_target_direction*).
2. **State 2: Pembacaan Target dan Rintangan Visual:** Jika tidak ada rintangan jarak dekat, robot berhenti diam selama 0.1 detik untuk menstabilkan gambar kamera dari efek kabur akibat guncangan (*motion blur*), memanggil REST API untuk mengambil

data visi terbaru, dan mengidentifikasi target terbesar dengan area > 600 piksel serta rintangan visual terbesar dengan area > 2000 piksel.

3. State 3: Eksekusi Gerakan Berprioritas:

- **Prioritas 1 (Pengejaran Target):** Jika target terdeteksi, robot melaju ke arah target menggunakan kendali kemudi PID yang dihitung dari koordinat target. Arah gerak disimpan ke variabel memori `last_known_target_direction` (LEFT atau RIGHT). Robot melaju selama 0.25 detik.
- **Prioritas 2 (Penghindaran Rintangan Visual):** Jika tidak ada target tetapi rintangan terdeteksi di area tengah (160 ± 75 piksel), robot menghindari dengan berbelok menjauh dari rintangan visual tersebut. Jika rintangan berada di luar area tengah, robot melaju lurus dengan aman. Robot melaju selama 0.25 detik.
- **Prioritas 3 (Pencarian Target berbasis Memori):** Jika tidak ada objek terdeteksi, robot memutar badannya ke arah memori target terakhir untuk menemukannya kembali selama 0.2 detik. Jika tidak ada memori arah target, robot diam demi keamanan.

```
1 import requests
2 import time
3 from .motor_manager import MotorManager
4 from .ultrasonic_manager import UltrasonicManager
5
6 #
7 # =====
8 #
9 # PARAMETER & KONFIGURASI
10 #
11 # --- Parameter PID (Digunakan untuk kalkulasi steer) ---
12 Kp = 0.8 # Lebih agresif
13 Ki = 0.0
14 Kd = 0.05
15
16 # --- Pengaturan Kecepatan Motor (PWM 0-255) ---
17 V_BASE = 125 # Diturunkan sedikit agar selisih kecepatan lebih terasa
18 V_SLOW = 100
19 MAX_SPEED_LIMIT = 180
20
21 # --- Konstanta Kamera ---
```

```

21 SETPOINT_TENGAH = 160
22
23 # --- Koneksi Visi ---
24 API_URL = "http://127.0.0.1:5000/api/objects"
25
26 #
27 # =====
28 #
29 # VARIABEL GLOBAL PID
30 #
31 # =====
32
33 last_error = 0
34 integral = 0
35
36 def hitung_PID(current_error):
37     global last_error, integral
38     P = current_error
39     integral += current_error
40     if integral > 50: integral = 50
41     if integral < -50: integral = -50
42     derivative = current_error - last_error
43     total_output = (Kp * P) + (Ki * integral) + (Kd * derivative)
44     last_error = current_error
45     return total_output
46
47 # --- Init Hardware ---
48 # Bias 1.14 untuk mengimbangi roda kiri secara permanen
49 motors = MotorManager(max_speed_limit=MAX_SPEED_LIMIT, left_bias=1.14)
50 ultrasonic = UltrasonicManager()
51
52 def jalankan_motor(v_kiri, v_kanan):
53     # Pastikan motor yang harusnya jalan punya torsi cukup (min 90)
54     if v_kiri > 0 and v_kiri < 90: v_kiri = 90
55     if v_kanan > 0 and v_kanan < 90: v_kanan = 90
56     if v_kiri < 0 and v_kiri > -90: v_kiri = -90
57     if v_kanan < 0 and v_kanan > -90: v_kanan = -90
58
59     motors.set_motors(v_kiri, v_kanan)
60
61 def stop_robot():
62     motors.stop()
63
64 def ambil_data_visi():
65     try:

```

```

62     r = requests.get(API_URL, timeout=0.1)
63     if r.status_code == 200:
64         return r.json()
65     except:
66         pass
67     return None
68
69 #
=====
70 # LOOP UTAMA (LOGIKA SEKUENSIAL: AMBIL GAMBAR -> JALAN -> STOP)
71 #
=====
72
73 def main_loop(shared_data=None):
74     SAFE_MARGIN_PIXELS = 100
75     MIN_AREA_TARGET = 600      # Abaikan target terlalu kecil/jauh
76     MIN_AREA_OBSTACLE = 2000   # Abaikan rintangan jauh
77     TARGET_REACHED_AREA = 10000 # Target dianggap "Penuh" di kamera (
    Finish)
78
79     print(f"Memulai Navigasi Sekuensial (Smooth | Bias: 1.18 | Body: 15
    cm) ...")
80
81     last_known_target_direction = "CENTER"
82
83     while True:
84         nav_aktif = shared_data.get('nav_active', False) if shared_data
            is not None else True
85
86         # 1. Baca Sensor Ultrasonik
87         jarak_mentah = ultrasonic.ukur_jarak_mentah()
88         if shared_data is not None:
89             if jarak_mentah is not None: shared_data['raw_distance'] =
                round(jarak_mentah, 2)
90             current_factor = shared_data.get('faktor_koreksi',
                ultrasonic.faktor_koreksi)
91             jarak_depan = (jarak_mentah * current_factor) if
                jarak_mentah else 999
92         else:
93             jarak_depan = ultrasonic.get_jarak()
94
95         if not nav_aktif:
96             stop_robot()

```

```

97         time.sleep(0.1)
98         continue
99
100     # --- STATE 1: JARAK DEKAT (MUNDUR & HINDAR) ---
101     if jarak_depan <= 15:
102         stop_robot()
103         time.sleep(0.1)
104
105     # Cek apakah ini target finish (Harus berwarna target DAN
106     area besar)
107     vision_data = ambil_data_visi()
108     target_area = 0
109     if vision_data:
110         for obj in vision_data.get('objects', []):
111             if obj['label'] == 'TARGET':
112                 area = obj['bbox']['w'] * obj['bbox']['h']
113                 if area > target_area: target_area = area
114
115     if target_area > TARGET_REACHED_AREA and jarak_depan <= 12:
116         print(f"[FINISH] Target Terpenuhi (Area: {target_area})
117         | Stop.")
118         stop_robot()
119         if shared_data is not None: shared_data['nav_active'] =
120         False
121         continue
122     else:
123         # Jika jarak dekat tapi bukan target penuh -> Itu
124         Obstacle
125         print(f">> OBSTACLE TERDETEKSI ({jarak_depan:.1f} cm) |
126         Manuver Mundur...")
127         jalankan_motor(-V_SLOW, -V_SLOW)
128         time.sleep(0.5)
129         stop_robot()
130         time.sleep(0.2)
131
132         # Belok sedikit untuk mencari jalan lain
133         if last_known_target_direction == "LEFT":
134             print(">> MANUEVER: Belok KANAN menghindari
135             rintangan")
136             jalankan_motor(120, -100)
137         else:
138             print(">> MANUEVER: Belok KIRI menghindari rintangan
139             ")
140             jalankan_motor(-100, 110)
141             time.sleep(0.3)

```

```

135         stop_robot()
136         time.sleep(0.2)
137         continue
138
139     # --- STATE 2: AMBIL GAMBAR & TENTUKAN ARAH ---
140     stop_robot() # Diam saat ambil gambar agar akurat
141     time.sleep(0.1)
142     vision_data = ambil_data_visi()
143
144     x_target = -1
145     x_obstacle = -1
146     if vision_data:
147         max_area_target = 0
148         max_area_obs = 0
149         for obj in vision_data.get('objects', []):
150             area = obj['bbox']['w'] * obj['bbox']['h']
151             if obj['label'] == 'TARGET' and area > MIN_AREA_TARGET:
152                 if area > max_area_target:
153                     max_area_target = area
154                     x_target = obj['bbox']['x'] + (obj['bbox']['w']
/ 2)
155             elif obj['label'] == 'OBSTACLE' and area >
MIN_AREA_OBSTACLE:
156                 if area > max_area_obs:
157                     max_area_obs = area
158                     x_obstacle = obj['bbox']['x'] + (obj['bbox']['w']
'] / 2)
159
160     # --- STATE 3: EKSEKUSI GERAKAN ---
161     # PRIORITAS 1: Jika ada target, kejar target!
162     if x_target != -1:
163         error = x_target - 160
164         steer = hitung_PID(error)
165
166         v_kiri = V_BASE + steer
167         v_kanan = V_BASE - steer
168
169     # Logging lebih sensitif (ambang batas diturunkan dari 40
ke 20)
170     if error < -20:
171         last_known_target_direction = "LEFT"
172         print(f">> BELOK KIRI: Target X:{x_target:.1f} | L:{
v_kiri:.1f} R:{v_kanan:.1f}")
173     elif error > 20:
174         last_known_target_direction = "RIGHT"

```

```

175         print(f">> BELOK KANAN: Target X:{x_target:.1f} | L:{
v_kiri:.1f} R:{v_kanan:.1f}")
176     else:
177         last_known_target_direction = "CENTER"
178         print(f">> MAJU: Target X:{x_target:.1f} | L:{v_kiri:.1
f} R:{v_kanan:.1f}")
179
180         jalankan_motor(v_kiri, v_kanan)
181         time.sleep(0.25)
182
183     # PRIORITAS 2: Jika tidak ada target tapi ada rintangan,
hindari!
184     elif x_obstacle != -1:
185         # Safe margin dipersempit agar tidak terlalu penakut (75px)
186         if x_obstacle < (160 + 75) and x_obstacle > (160 - 75):
187             if x_obstacle < 160:
188                 print(f">> HINDAR KANAN: Ada rintangan di kiri")
189                 jalankan_motor(V_BASE + 35, V_BASE - 35)
190             else:
191                 print(f">> HINDAR KIRI: Ada rintangan di kanan")
192                 jalankan_motor(V_BASE - 25, V_BASE + 25)
193         else:
194             print(f">> MAJU: Rintangan di luar jalur (Aman)")
195             jalankan_motor(V_BASE, V_BASE)
196             time.sleep(0.25)
197
198     else:
199         # PRIORITAS 3: Cari target berdasarkan memori
200         if last_known_target_direction == "LEFT":
201             print(f">> SEARCH: Putar KIRI mencari target...")
202             jalankan_motor(-100, 110)
203         elif last_known_target_direction == "RIGHT":
204             print(f">> SEARCH: Putar KANAN mencari target...")
205             jalankan_motor(120, -100)
206         else:
207             print(f">> SEARCH: Target hilang, robot diam.")
208             stop_robot()
209             time.sleep(0.2)
210
211     stop_robot()
212     time.sleep(0.1) # Jeda stabilisasi
213
214 if __name__ == "__main__":
215     try:
216         main_loop()

```

```
217     except KeyboardInterrupt:
218         stop_robot ()
219     finally:
220         motors.cleanup ()
```

Listing 8: Kode Program Modul Navigasi Utama (navigator.py)

4 Pengujian dan Hasil Pembahasan

Pengujian dilakukan untuk mengukur keandalan sistem robotika yang telah dirancang. Pengujian mencakup kalibrasi akurasi sensor jarak serta pengujian navigasi pada lintasan uji coba.

4.1 Pengujian Akurasi Sensor Jarak

Pengujian sensor dilakukan dengan membandingkan nilai jarak pembacaan sensor ultrasonik dengan jarak pengukuran manual menggunakan penggaris. Hasil pengujian ditunjukkan pada Tabel 1.

Tabel 1: Data Pengujian Akurasi Sensor Jarak Ultrasonik HC-SR04.

No.	Jarak Aktual (cm)	Jarak Sensor (cm)	Selisih / Error (cm)	Persentase Error (%)
1	10.0	10.1	0.1	1.00
2	20.0	19.8	0.2	1.00
3	30.0	30.3	0.3	1.00
4	40.0	39.5	0.5	1.25
5	50.0	50.2	0.2	0.40
Rata-rata Error (%)				0.93%

Berdasarkan Tabel 1, terlihat bahwa sensor ultrasonik memiliki rata-rata error sebesar 0.93%, yang mana masih berada di bawah ambang batas toleransi sistem kendali (5%), sehingga sensor dinyatakan layak digunakan untuk sistem navigasi robot.

4.2 Analisis Pengujian Navigasi

Setelah dilakukan kalibrasi sensor, pengujian dilanjutkan dengan mengamati respon robot dalam mendeteksi penghalang di depannya. Robot berhasil mendeteksi dinding penghalang pada jarak ≈ 20 cm dan melakukan manuver belok kanan secara otomatis untuk menghindari tabrakan. Eksperimen dilakukan sebanyak 10 kali lintasan dengan persentase keberhasilan navigasi sebesar 90%.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem robotika yang telah dilakukan, dapat diambil beberapa kesimpulan sebagai berikut:

1. Prototipe robot berhasil dirancang dengan mengintegrasikan sensor ultrasonik HC-SR04, driver motor L298N, dan mikrokontroler Arduino Uno.
2. Hasil pengujian sensor menunjukkan tingkat keandalan yang tinggi dengan rata-rata error sebesar 0.93%.
3. Logika kendali kecerdasan buatan sederhana (*obstacle avoidance*) berhasil memandu robot menghindari halangan dengan tingkat keberhasilan sebesar 90% dari total pengujian.

5.2 Saran

Beberapa saran yang dapat diajukan untuk pengembangan sistem robot ini lebih lanjut adalah:

1. Menambahkan sensor jarak di bagian samping kiri dan kanan robot agar kemampuan pemetaan lingkungan sekitar menjadi lebih akurat.
2. Mengimplementasikan sistem kendali yang lebih cerdas seperti Fuzzy Logic Controller untuk menghasilkan pergerakan motor yang lebih halus (*smooth*).
3. Menggunakan baterai dengan kapasitas daya yang lebih besar untuk memperpanjang waktu pengujian robot di lapangan.

Daftar Pustaka

- [1] Yu Liu et al. “A Review of Sensing Technologies for Indoor Autonomous Mobile Robots”. In: *Sensors* 24.4 (2024), p. 1222. DOI: [10.3390/s24041222](https://doi.org/10.3390/s24041222).
- [2] Shubo Wang et al. “A Monocular Vision Obstacle Avoidance Method Applied to Indoor Tracking Robot”. In: *Drones* 5.4 (2021), p. 105. DOI: [10.3390/drones5040105](https://doi.org/10.3390/drones5040105).
- [3] Thai Viet Dang and Ngoc Tam Bui. “Obstacle Avoidance Strategy for Mobile Robot Based on Monocular Camera”. In: *Electronics* 12.8 (2023), p. 1932. DOI: [10.3390/electronics12081932](https://doi.org/10.3390/electronics12081932).
- [4] Yingrui Jin et al. “Target Recognition and Navigation Path Optimization Based on NAO Robot”. In: *Applied Sciences* 12.17 (2022), p. 8466. DOI: [10.3390/app12178466](https://doi.org/10.3390/app12178466).
- [5] Jawad N. Yasin et al. “Low-Cost Ultrasonic Based Object Detection and Collision Avoidance Method for Autonomous Robots”. In: *International Journal of Information Technology* 13 (2021), pp. 97–107. DOI: [10.1007/s41870-020-00513-w](https://doi.org/10.1007/s41870-020-00513-w).
- [6] Hongzhou Yan et al. “An Obstacle Avoidance Algorithm for Unmanned Surface Vehicle Based on A Star and Velocity-Obstacle Algorithms”. In: *2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC)*. Vol. 6. 2022, pp. 77–82. DOI: [10.1109/ITOEC53115.2022.9734642](https://doi.org/10.1109/ITOEC53115.2022.9734642).
- [7] Ziming Zhang et al. “Dynamic Obstacle Avoidance Path Planning of Unmanned Vehicle Based on Improved APF”. In: *2023 7th International Symposium on Computer Science and Intelligent Control (ISCSIC)*. 2023, pp. 135–140. DOI: [10.1109/ISCSIC60498.2023.00037](https://doi.org/10.1109/ISCSIC60498.2023.00037).
- [8] Ji Chang Kim, Dong Sung Pae, and Myo Taeg Lim. “Obstacle Avoidance Path Planning Algorithm Based on Model Predictive Control”. In: *2018 18th International Conference on Control, Automation and Systems (ICCAS)*. 2018, pp. 141–143.